

COURS TD TP 12 JAVA

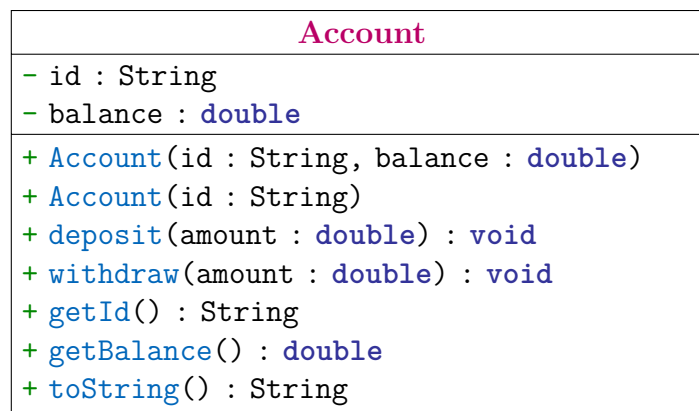
Avant de commencer, pensez à créer le projet Eclipse `CoursJava` dans lequel vous enregistrerez les packages correspondant aux différents exercices.

L'objectif de cet exercice est de passer en revue les nombreuses notions que nous avons abordées jusqu'à présent. Pour cela, nous allons écrire une application permettant de gérer une banque. Nous devons représenter un compte client, un client et une banque.

Q1. Créez sous Eclipse le package `tp.tp12` pour y placer vos fichiers.

Account

Q2. Écrivez une classe `Account`, dont chaque instance représentera le compte d'un client. Pour implémenter cette classe, appuyez-vous sur le diagramme de classe UML suivant :

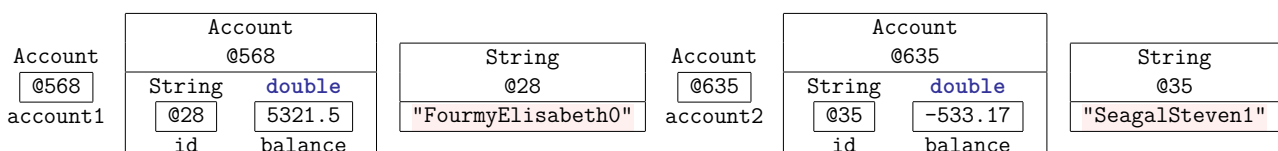


Utilisez également les remarques suivantes :

- La propriété `id` représente l'identifiant du compte ;
- La propriété `balance` représente le solde du compte, qui peut être négatif ;
- Le constructeur `Account(String id, double balance)` initialise les propriétés à partir des valeurs des paramètres ;
- Le constructeur `Account(String id)` initialise l'identifiant à partir de la valeur en paramètre et le solde à zéro. Dans ce constructeur, n'oubliez pas de déléguer l'initialisation à l'autre constructeur ;
- La méthode `void deposit(double amount)` permet de déposer le montant `amount` sur le compte. On supposera que ce montant est toujours positif, il n'est donc pas nécessaire de le vérifier ;
- La méthode `void withdraw(double amount)` permet de retirer le montant `amount` du compte. On supposera que ce montant est toujours positif, il n'est donc pas nécessaire de le vérifier ;
- La méthode `String toString()` retourne une chaîne représentant le compte. Par exemple, si l'identifiant du compte est `"FourmyElisabeth0"` et le solde est de `5321.15`, alors la chaîne retournée sera :

`"Compte FourmyElisabeth0 - Solde 5321.15"`

Q3. Écrivez le code de la classe `TestAccount` permettant d'afficher deux comptes dont vous pouvez déduire le contenu à partir de l'état de la mémoire suivant :



Q4. Comment l'encapsulation garantit-elle que l'identifiant du compte reste inchangé après sa création ?

Q5. Ajoutez la méthode `equals` à la classe `Account`, permettant de comparer deux comptes. Deux comptes sont considérés comme égaux s'ils possèdent le même identifiant, même si leurs soldes sont différents.

Q6. Écrivez la classe `TestAccountEquals` qui demande à l'utilisateur de saisir l'identifiant et le solde d'un premier compte, puis ceux d'un second compte. La classe doit ensuite afficher le message `"identiques"` ou `"différents"` pour indiquer si les deux comptes sont identiques ou non, en utilisant la méthode `equals` de la classe `Account`. Exécutez ce test en saisissant deux comptes avec le même identifiant, puis deux comptes avec des identifiants différents. Remarque : Une erreur classique consiste à comparer les références des identifiants au lieu de comparer leur contenu. Assurez-vous d'utiliser la méthode adéquate pour comparer les chaînes de caractères.

Q7. Créez la classe utilitaire `BankUtils` qui va contenir des méthodes `static` pour manipuler les différentes classes que vous allez définir. Même si cela n'est pas précisé, toutes les méthodes de cette classe devront être publiques et `static`. Créez également une classe exécutable `TestBankUtils` que vous utiliserez pour tester les différentes méthodes de la classe `BankUtils`.

Q8. Ajoutez à la classe `BankUtils` une méthode publique statique permettant de créer un compte à partir d'un nom, d'un prénom, d'un indice de compte et d'un solde maximal. L'identifiant du compte devra être construit en concaténant le nom, le prénom et l'indice, et le solde du compte devra être initialisé à une valeur aléatoire comprise entre `-maxBalance` et `maxBalance`.

```
Account createAccount(String lastName, String firstName, int index,
                      double maxBalance)
```

Par exemple, si le nom est `"Fourmy"`, le prénom est `"Elisabeth"` et l'indice est `3`, alors l'identifiant du compte sera `"FourmyElisabeth3"`, et le solde sera une valeur aléatoire entre `-maxBalance` et `maxBalance`. Ajoutez un test de cette méthode dans la classe `TestBankUtils` afin de vérifier que les comptes sont correctement créés avec un identifiant et un solde respectant les contraintes spécifiées.

Q9. Ajoutez à la classe `BankUtils` une méthode publique statique permettant d'afficher une liste de comptes. Cette méthode devra prendre en paramètre une liste de comptes et afficher chaque compte sur une ligne.

```
void printAccounts(ArrayList<Account> accounts)
```

Ajoutez un test de cette méthode dans la classe `TestBankUtils` afin de vérifier que les comptes sont correctement affichés.

Q10. Ajoutez à la classe `BankUtils` une méthode publique statique permettant de créer une liste de comptes à partir d'un nom, d'un prénom, d'un nombre de comptes et d'un solde maximal. Chaque compte devra avoir un identifiant unique construit en concaténant le nom, le prénom et un indice allant de `0` à `numberOfAccounts - 1`. Le solde de chaque compte devra être initialisé à une valeur aléatoire comprise entre `-maxBalance` et `maxBalance`.

```
ArrayList<Account> createAccounts(String lastName, String firstName,
                                  int numberOfAccounts, double maxBalance)
```

Par exemple, si le nom est `"Fourmy"`, le prénom est `"Elisabeth"`, le nombre de comptes est `3` et le solde maximal est `5000`, alors la méthode devra retourner une liste contenant les comptes suivants :

- `"FourmyElisabeth0"` avec un solde aléatoire entre `-5000` et `5000`
- `"FourmyElisabeth1"` avec un solde aléatoire entre `-5000` et `5000`
- `"FourmyElisabeth2"` avec un solde aléatoire entre `-5000` et `5000`

Ajoutez un test de cette méthode dans la classe `TestBankUtils` afin de vérifier que la liste de comptes est correctement créée avec des identifiants uniques et des soldes respectant les contraintes spécifiées.

Q11. Dans les méthodes `printAccounts` et `createAccounts`, remplacez `ArrayList<Account>` par `List<Account>`. Expliquez pourquoi vos tests fonctionnent toujours malgré ce changement de type de retour. Quel est l'intérêt d'utiliser `List<Account>` au lieu de `ArrayList<Account>` ?

Q12. Ajoutez à la classe `BankUtils` une méthode publique statique permettant d'appliquer des frais fixes à l'ensemble des comptes d'une liste. Cette méthode devra prendre en paramètre une liste de comptes ainsi que le montant des frais à appliquer.

```
void applyFees(List<Account> accounts, double feeAmount)
```

Utilisez également les remarques suivantes :

- La méthode parcourt la liste et soustrait le montant des frais au solde de chaque compte ;
- Les frais sont simplement appliqués, sans vérification particulière ;

Ajoutez un test de cette méthode dans la classe `TestBankUtils`. Les tests consistent à :

- Créer une liste de comptes ;
- Afficher la liste de comptes ;
- Appliquer une valeur fixe de frais à cette liste ;
- Afficher à nouveau la liste de comptes afin de vérifier que les frais ont bien été appliqués.

Q13. Ajoutez à la classe `BankUtils` une méthode publique statique permettant d'appliquer un bonus à l'ensemble des comptes d'une liste. Cette méthode devra prendre en paramètre une liste de comptes ainsi qu'un pourcentage de bonus à appliquer.

```
void applyBonus(List<Account> accounts, double bonusPercentage)
```

Utilisez également les remarques suivantes :

- La méthode parcourt la liste et applique un bonus uniquement aux comptes ayant un solde strictement positif ;
- Le bonus est calculé en multipliant le solde par le pourcentage spécifié, puis ajouté au solde du compte ;

Ajoutez un test de cette méthode dans la classe `TestBankUtils`. Les tests consistent à :

- Créer une liste de comptes avec des soldes variés ;
- Afficher la liste de comptes ;
- Appliquer un bonus en pourcentage à cette liste en utilisant la méthode `applyBonus` ;
- Afficher à nouveau la liste de comptes afin de vérifier que le bonus a bien été appliqué.

Q14. Le comportement des méthodes `applyFees` et `applyBonus` est très similaire : elles appliquent un traitement à l'ensemble des comptes d'une liste. Afin d'éviter cette répétition de code, nous allons définir une interface permettant de généraliser ces traitements. Définissez une interface `AccountProcessor` contenant la méthode `void process(Account account)` permettant d'appliquer un traitement à un compte. Cette interface sera utilisée pour généraliser les traitements appliqués aux comptes.

Q15. Définissez la classe `FeeProcessor`, qui est une implémentation de l'interface `AccountProcessor` et qui applique des frais de 100 euros à un compte. Créez ensuite une classe `TestAccountProcessor` pour vérifier que `FeeProcessor` applique correctement les frais à un compte. Pour cela, créez un compte puis affichez-le. Ensuite, créez un `FeeProcessor` et exécutez-le sur le compte, puis affichez à nouveau le compte.

Q16. Modifiez la classe `FeeProcessor` afin d'ajouter une propriété `feeAmount` permettant de définir dynamiquement le montant des frais à appliquer à un compte et de ne pas être figé à 100. Ajoutez un constructeur permettant d'initialiser cette valeur lors de la création d'une instance de `FeeProcessor`.

Mettez à jour la classe `TestAccountProcessor` pour tester cette nouvelle version de `FeeProcessor` en appliquant des frais de 50 euros.

Q17. Créez la classe `BonusProcessor`, qui est une implémentation de l'interface `AccountProcessor` et qui applique un bonus à un compte si son solde est strictement positif. Ce bonus devra être défini dynamiquement à l'aide d'une propriété `bonusPercentage`. Ajoutez un constructeur permettant d'initialiser cette valeur lors de la création d'une instance de `BonusProcessor`. Ajoutez à la classe `TestAccountProcessor` un test pour vérifier que `BonusProcessor` applique correctement un bonus de 3 %.

Avant de continuer, prenez le temps de comprendre la démarche que nous sommes en train de mettre en uvre et prendre du recul. Nous avons débuté avec les méthodes `applyFees` et `applyBonus`, permettant d'appliquer un traitement sur chaque compte d'une liste de comptes. Comme ces deux méthodes sont très similaires, nous avons cherché à éviter la duplication de code en introduisant une approche plus générique.

Pour cela, nous avons défini une interface `AccountProcessor`, qui impose l'implémentation de la méthode `process(Account account)`, permettant d'appliquer un traitement à un compte. Ensuite, nous avons créé deux implémentations spécifiques :

- `FeeProcessor`, qui applique des frais à un compte en soustrayant un montant fixe défini dynamiquement ;
- `BonusProcessor`, qui applique un bonus sous forme d'un pourcentage du solde uniquement si celui-ci est strictement positif.

Ces implémentations permettent donc d'isoler les traitements appliqués à un compte dans des classes spécifiques, rendant ainsi le code plus modulaire, plus extensible et plus facilement réutilisable.

Q18. Ajoutez à la classe `BankUtils` la méthode `applyToAll` qui applique un traitement défini par un objet implémentant `AccountProcessor` à chaque compte d'une liste.

```
public static void applyToAll(List<Account> accounts, AccountProcessor processor)
```

Ajoutez un test dans la classe `TestAccountProcessor` pour vérifier que la méthode `applyToAll` fonctionne correctement avec les classes `FeeProcessor` et `BonusProcessor`.

Nous avons ainsi achevé la démarche permettant d'éviter la duplication de code entre les méthodes `applyFees` et `applyBonus`. L'introduction de l'interface `AccountProcessor` et de la méthode générique `applyToAll` a permis de généraliser le traitement des comptes bancaires, rendant le code plus modulaire et réutilisable.

Customer

Q19. Écrivez une classe `Customer`, dont chaque instance représentera un client et ses comptes. Pour implémenter cette classe, appuyez-vous sur le diagramme UML suivant :

Customer	
-	lastName : String
-	firstName : String
-	accounts : List<Account>
+	Customer(lastName : String, firstName : String)
+	getAccountsBalance() : double
+	addAccount() : void
-	getAccount(accountId : String) : Account
+	deposit(accountId : String, amount : double) : boolean
+	withdraw(accountId : String, amount : double) : boolean
+	getLastName() : String
+	getFirstName() : String
+	toString() : String

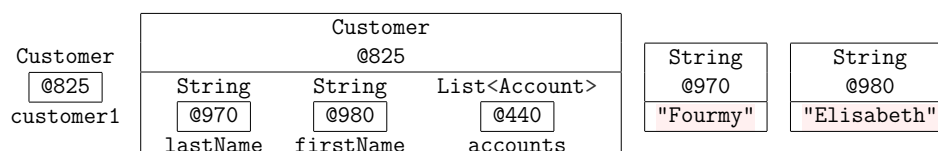
Utilisez également les remarques suivantes :

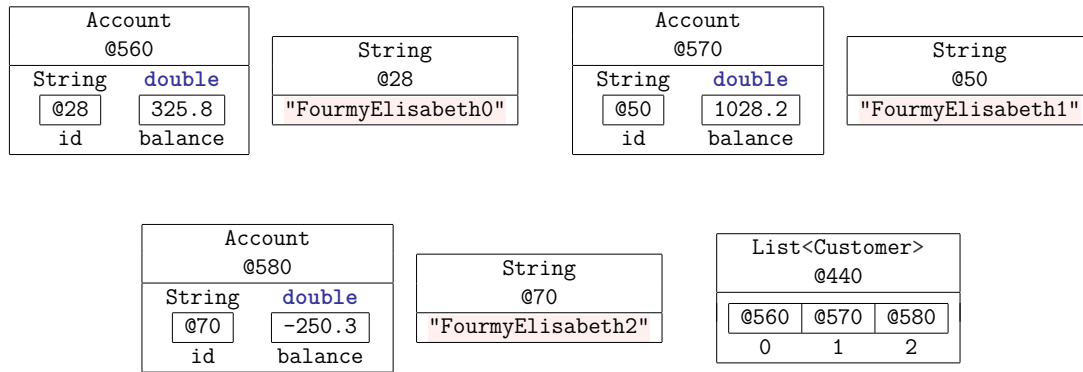
- La propriété `lastName` représente le nom du client ;
- La propriété `firstName` représente le prénom du client ;
- La propriété `accounts` est une liste contenant les comptes bancaires du client ;
- Le constructeur `Customer(String lastName, String firstName)` initialise les propriétés et crée une liste vide de comptes ;
- La méthode `double getAccountsBalance()` calcule et retourne le solde total des comptes du client ;
- La méthode `void addAccount()` permet de créer un nouveau compte et de l'ajouter à la liste des comptes du client. L'identifiant du compte est construit en concaténant le nom, le prénom et le nombre actuel de comptes du client. Le solde du compte est initialisé à 0.
- La méthode `Account getAccount(String accountId)` recherche un compte à partir de son identifiant. Si le compte est trouvé, il est retourné ; sinon, la méthode retourne `null`.
- La méthode `boolean deposit(String accountId, double amount)` permet d'effectuer un dépôt sur un compte du client en utilisant son identifiant. Si le compte existe, le montant est ajouté à son solde et la méthode retourne `true`. Si le compte n'existe pas, elle retourne `false`.
- La méthode `boolean withdraw(String accountId, double amount)` permet d'effectuer un retrait sur un compte du client en utilisant son identifiant. Si le compte existe, le montant est soustrait de son solde et la méthode retourne `true`. Si le compte n'existe pas, elle retourne `false`.
- La méthode `String toString()` retourne une chaîne représentant le client et ses comptes.

Pour compléter le corps de la méthode `toString()`, appuyez-vous sur l'exemple suivant, obtenu après l'affichage d'un client `Seagal Steven` auquel on a ajouté trois comptes :

```
Seagal Steven
Compte StevenSeagal0 - Solde 0.0
Compte StevenSeagal1 - Solde 0.0
Compte StevenSeagal2 - Solde 0.0
Solde total : 0.0
```

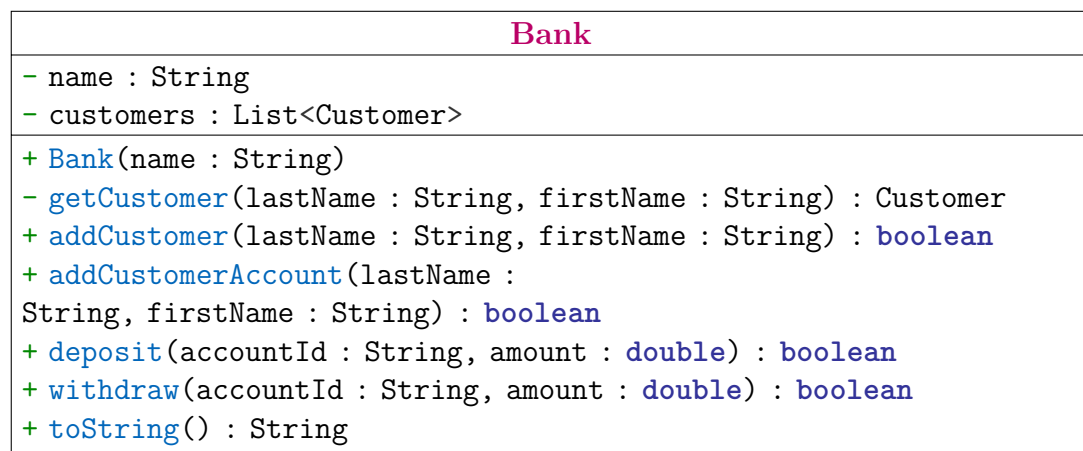
Q20. Écrivez une classe `TestCustomer` qui permet de construire, d'initialiser et d'afficher un client dont l'état de la mémoire peut être représenté de la manière suivante :





Bank

Q21. Écrivez une classe `Bank`, dont chaque instance représentera une banque. Pour implémenter cette classe, appuyez-vous sur le diagramme de classe UML suivant :



Utilisez également les remarques suivantes :

- La propriété `name` représente le nom de la banque ;
- La propriété `customers` est une liste de clients de la banque sous forme d'un `List<Customer>` ;
- Le constructeur `Bank(String name)` initialise le nom de la banque et une liste vide de clients ;
- La méthode `Customer getCustomer(String lastName, String firstName)` recherche un client par son nom et prénom et retourne `null` s'il n'existe pas ;
- La méthode `boolean addCustomer(String lastName, String firstName)` ajoute un nouveau client s'il n'existe pas déjà. Elle retourne `true` en cas de succès et `false` si le client existe déjà ;
- La méthode `boolean addCustomerAccount(String lastName, String firstName)` ajoute un compte à un client existant. Elle retourne `true` si l'ajout est réussi et `false` si le client n'existe pas ;
- La méthode `boolean deposit(String accountId, double amount)` permet de déposer une somme sur un compte existant. Retourne `true` si l'opération est réalisée et `false` si le compte n'existe pas ;
- La méthode `boolean withdraw(String accountId, double amount)` permet de retirer une somme d'un compte existant. Retourne `true` si l'opération est réalisée et `false` si le compte n'existe pas ;
- La méthode `String toString()` retourne une chaîne représentant la banque.

Pour compléter le corps de la méthode `toString()` et réaliser vos tests, écrivez une classe `TestBank` contenant une méthode `main` permettant de tester les différentes fonctionnalités de la classe `Bank`. Assurez-vous que l'affichage obtenu correspond bien à l'exemple suivant :

Nom de la banque : L'écureuil de Luminy

Liste des clients

CLIENT 0

Fourmy Elisabeth

Compte FourmyElisabeth0 - Solde 5321.15

Compte FourmyElisabeth1 - Solde 2533.17

Solde total : 7854.32

CLIENT 1

Seagal Steven

Compte SeagalSteven0 - Solde 345.15

Compte SeagalSteven1 - Solde -533.17

Compte SeagalSteven2 - Solde -799.33

Solde total : -987.35

Q22. Ajoutez à la classe `Bank` une méthode `applyProcessor(AccountProcessor processor)` permettant d'appliquer un traitement à l'ensemble des comptes de la banque.

Pour cela, ajoutez également la méthode `applyProcessor(AccountProcessor processor)` à la classe `Customer`, afin d'appliquer un traitement à tous les comptes appartenant à un client.

Enfin, dans la classe `TestBank`, testez votre implémentation en appliquant successivement :

- Des frais fixes à tous les comptes,
- Un bonus aux comptes ayant un solde positif.

Affichez l'état des comptes avant et après chaque traitement pour observer l'impact des opérations sur le solde des clients.